SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
PRACTICAL ASSIGNMENT TEMPLATE

1. Student Details

Student Name	Pratik Akrur Pandit
PRN	202501110116
Division	B
Batch	B2
Roll No.	202501110116
Date of Submission	27/10/2026

2. Practical Details

Practical No.	12
Practical Title	Type Casting and Casting Operators
Date Performed	24/10/2026
Faculty Name	Shruti Gadewar

3. Problem Statement

Write a C++ program based on type casting and casting operators. The program must demonstrate the four casting operators of C++ – static_cast, dynamic_cast, const_cast and reinterpret_cast – and show how each one converts a value or a pointer from one type to another.

4. Objectives

1. To understand type casting and the need for casting operators in C++.
2. To use static_cast, dynamic_cast, const_cast and reinterpret_cast in programs.
3. To understand the safety checks and risks associated with each casting operator.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Convert data from one type to another using the appropriate casting operator.
2. Perform safe down casting using dynamic_cast and handle a failed cast.
3. Explain the difference between the four casting operators and when each should be used.

6. Theory / Concept

Explain the important concepts, algorithms, techniques, or methods used.

Type casting means converting a value from one data type to another. **Casting operators** are used for type casting in C++; they convert one data type to another in a controlled and visible way. C++ supports four types of casts: static_cast, dynamic_cast, const_cast and reinterpret_cast.

A. static_cast

- The most commonly used casting operator in C++. It performs a compile-time type conversion and is mainly used for explicit conversions that the compiler considers safe.
- It can be used to convert between related types, such as numeric types, or pointers in the same inheritance hierarchy.
- Syntax: **static_cast <new_type> (expression);** where expression is the data to be converted and new_type is the desired type.

B. dynamic_cast

- Mainly used to perform down casting (converting a pointer or reference of a base class to a derived class).
- It ensures type safety by performing a runtime check to verify that the conversion is valid; the base class must be polymorphic (have a virtual function).
- If the conversion is not possible, it returns a null pointer (for pointer conversions) or throws a bad_cast exception (for reference conversions).
- Syntax: **dynamic_cast <new_type> (expression);**

C. const_cast

- Used to modify the const or volatile qualifier of a variable. It allows the programmer to temporarily remove the constancy of an object and make modifications.
- Caution must be exercised, as modifying an object that was actually declared const can lead to undefined behavior.
- Syntax: **const_cast <new_type> (expression);**

D. reinterpret_cast

- Used to convert a pointer to any other type of pointer.
- It does not check whether the pointer converted is of the same type or not, so it must be used carefully.
- Syntax: **reinterpret_cast <new_type> (expression);**

7. Algorithm / Pseudocode

Write the step-by-step algorithm or pseudocode for the proposed solution.

START

Step 1: Define a polymorphic class Base (with a virtual function) and a class Derived that inherits from it.

Step 2: static_cast: divide two integers with and without casting to double, convert double to int, char to int and int to char, and down cast a Base pointer to a Derived pointer.

Step 3: dynamic_cast: cast a Base pointer holding a Derived object (succeeds) and a Base pointer holding a Base object (returns NULL); also cast a reference and catch the bad_cast exception.

Step 4: const_cast: remove the const qualifier from a pointer using const_cast and modify the value through it.

Step 5: reinterpret_cast: convert an int pointer to a char pointer, display the first byte as a character, and then display all the results.

STOP

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	C++
IDE / Platform	Dev-C++ (Windows)

Libraries / Packages	<iostream> (input/output), <typeinfo> (bad_cast exception)

09. Implementation / GitHub Repository

GitHub Repository URL	https://github.com/panditpratik/OOP-
Repository Name	**Problem-Solving-Using-OOP-in-Cpp**
Branch Used	main
Commit ID / Latest Commit	0ae88150a210e316a425746b2cb833bb87f37e91
Date of Last Commit	27/10/26

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
TC1 static_cast int / int	7 / 2 (no cast)	3	3	Pass
TC2 static_cast to double	static_cast<double>(7) / 2	3.5	3.5	Pass
TC3 static_cast double to int	9.99	9	9	Pass
TC4 static_cast char to int	'A'	65	65	Pass
TC5 static_cast int to char	97	a	a	Pass
TC6 static_cast down cast	Base* to Derived*	Derived-only function called	Derived-only function called	Pass
TC7 dynamic_cast (valid)	Base* holding a Derived object	Cast of b1 succeeded: Derived-only function called	Cast of b1 succeeded: Derived-only function called	Pass
TC8 dynamic_cast (invalid)	Base* holding a Base object	Cast of b2 failed: dynamic_cast returned NULL	Cast of b2 failed: dynamic_cast returned NULL	Pass
TC9 dynamic_cast reference	*b2 as Derived&	bad_cast exception caught	bad_cast exception caught	Pass
TC10 const_cast	val = 50, then *ncPtr = 100	After const_cast : val = 100	After const_cast : val = 100	Pass
TC11 reinterpret_cast	int 65 viewed as char*	A (ASCII 65)	A (ASCII 65)	Pass

For ML practicals, include appropriate performance metrics, graphs, tables, or model outputs wherever applicable.

11. Results / Observations

Mention the important results obtained during execution.

- static_cast changed the result of the division from 3 to 3.5 by converting an operand to double, and it also converted double to int (9.99 became 9) and between char and int.
- dynamic_cast succeeded when the base pointer really pointed to a Derived object, and returned NULL (pointer cast) or threw bad_cast (reference cast) when it pointed to a Base object.
- const_cast removed the const qualifier of a pointer, which allowed the value of val to be changed from 50 to 100.
- reinterpret_cast let the memory of an int be read as a char without any check; the first byte of the int 65 was read as the character 'A' (the result depends on the byte order of the machine).

Screenshots / Graphs / Output:

Practical_assignment_12.cpp > main()

```cpp
#include <iostream>
#include <typeinfo>
using namespace std;

class Base
{
public:
    virtual void show()
    {
        cout << "Base class show()" << endl;
    }

    virtual ~Base() {}
};

class Derived : public Base
{
public:
    void show()
    {
        cout << "Derived class show()" << endl;
    }

    void derivedOnly()
    {
        cout << "Derived-only function called" << endl;
    }
};

int main()
{
    // ================= 1. static_cast =================

    cout << "===== 1. static_cast =====" << endl;

    int a = 7, b = 2;

    cout << "Without cast : 7 / 2 = " << (a / b) << endl;
    cout << "With cast    : 7 / 2 = "
         << static_cast<double>(a) / b << endl;

    double d = 9.99;
    int i = static_cast<int>(d);

    cout << "double 9.99 to int : " << i << endl;

    char ch = 'A';
```

```
    cout << "char 'A' to int : " << static_cast<int>(ch) << endl;

    int n = 97;

    cout << "int 97 to char : " << static_cast<char>(n) << endl;

    Derived dObj;
    Base *basePtr = &dObj;

    Derived *dPtr = static_cast<Derived*>(basePtr);
    // Downcast in the same hierarchy

    dPtr->derivedOnly();

    // ================ 2. dynamic_cast ================

    cout << endl << "===== 2. dynamic_cast =====" << endl;

    Base *b1 = new Derived();
    // Base pointer holding a Derived object

    Base *b2 = new Base();
    // Base pointer holding a Base object

    Derived *d1 = dynamic_cast<Derived*>(b1);

    if (d1 != NULL)
    {
        cout << "Cast of b1 succeeded: ";
        d1->derivedOnly();
    }
    else
    {
        cout << "Cast of b1 failed" << endl;
    }

    Derived *d2 = dynamic_cast<Derived*>(b2);

    if (d2 != NULL)
    {
        cout << "Cast of b2 succeeded" << endl;
    }
    else
    {
        cout << "Cast of b2 failed: dynamic_cast returned NULL" << endl;
    }

```

```
    try
    {
        Derived &ref = dynamic_cast<Derived&>(*b2);
        ref.derivedOnly();
    }
    catch (bad_cast &)
    {
        cout << "Reference cast of *b2 failed: bad_cast exception caught"
             << endl;
    }

    delete b1;
    delete b2;

    // ================ 3. const_cast ================

    cout << endl << "===== 3. const_cast =====" << endl;

    int val = 50;

    const int *cPtr = &val;
    // Pointer to const int

    cout << "Before const_cast : val = " << val << endl;

    int *ncPtr = const_cast<int*>(cPtr);
    // Remove constness from the pointer type

    *ncPtr = 100;

    cout << "After const_cast : val = " << val << endl;

    // ================ 4. reinterpret_cast ================

    cout << endl << "===== 4. reinterpret_cast =====" << endl;

    int num = 65;

    int *iPtr = &num;

    char *cp = reinterpret_cast<char*>(iPtr);
    // View the int's memory through a char pointer

    cout << "Value of int        : " << num << endl;

    cout << "First byte as char : " << *cp
         << " (ASCII " << static_cast<int>(*cp) << ")" << endl;
```

```
    cout << "First byte as char : " << *cp
         << " (ASCII " << static_cast<int>(*cp) << ")" << endl;


    return 0;
}
```

- Output:-

```
PS D:\All Programs\OOP(C++)> cd "d:\All Programs\OOP(C++)\" ;
===== 1. static_cast =====
Without cast : 7 / 2 = 3
With cast    : 7 / 2 = 3.5
double 9.99 to int : 9
char 'A' to int : 65
int 97 to char : a
Derived-only function called

===== 2. dynamic_cast =====
Cast of b1 succeeded: Derived-only function called
Cast of b2 failed: dynamic_cast returned NULL
Reference cast of *b2 failed: bad_cast exception caught

===== 3. const_cast =====
Before const_cast : val = 50
After const_cast : val = 100

===== 4. reinterpret_cast =====
Value of int        : 65
First byte as char : A (ASCII 65)
PS D:\All Programs\OOP(C++)>
```

12. Conclusion

Write a brief conclusion describing what was implemented and what was learned.

In this practical, a C++ program was implemented to demonstrate the four casting operators: static_cast, dynamic_cast, const_cast and reinterpret_cast. I learned that static_cast is used for safe compile-time conversions, dynamic_cast gives a safe runtime-checked down cast, const_cast removes the const qualifier and reinterpret_cast converts one pointer type to another without any check, so it should be used with care.

13. Assignment Questions

Q1. What is type casting? Name the casting operators supported by C++.

Answer: Type casting is the conversion of a value from one data type to another. C++ provides casting operators for this purpose, and it supports four types of casts: **static_cast**, **dynamic_cast**, **const_cast** and **reinterpret_cast**. Each one has the syntax *cast_operator <new_type> (expression);*, where expression is the data to be converted and new_type is the desired type.

Q2. Differentiate between static_cast and dynamic_cast.

Basis	**static_cast**	**dynamic_cast**
Type checking	Done at compile time	Done at runtime
Main use	Conversions between related types (numeric types, pointers in the same hierarchy)	Safe down casting from a base class pointer/reference to a derived class
Safety	No runtime check; the programmer must make sure the conversion is valid	Verifies the validity of the conversion at runtime

Basis	static_cast	dynamic_cast
On failure	No check, so wrong use gives undefined behavior	Returns NULL (pointers) or throws bad_cast (references)
Requirement	None	Base class must be polymorphic (have a virtual function)

14. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Pratik Akrur Pandit	202501110116	